
NovaMart: A Causally Consistent Simulated Enterprise for Measuring Tribal Knowledge Extraction

Susnato Dhar
Pavo AI, Bangalore
susnato@pavoai.com

Srijan Saket
Pavo AI, San Francisco
srijan@pavoai.com

Rishabh Mehrotra
Pavo AI, London
rishabh@pavoai.com

Abstract

AI agents are moving into the daily work of enterprise engineers and analysts: building pipelines, answering metric questions, changing production code. Doing any of this well requires the company’s *tribal knowledge*: which undocumented filters go into a metric, which of several views is canonical, which half-finished migration decided whose data moved. Agents that lack it fail silently, shipping numbers that look right while disagreeing with every existing report. Evaluating this capability is hard because the knowledge is spread across adjacent sources (codebase, commit history, databases, deployment logs, historical queries, dashboards), artifact classes companies share, if at all, only as stripped metadata; as of our survey (July 2026), no benchmark covers them together. We present **NovaMart**: a simulated e-commerce company covering all of these sources mentioned earlier, causally connected because its entire history is *executed rather than authored* on real behavioral traffic; the full estate is released with 51 audited tribal-knowledge claims. Evaluating three widely used agent systems, we find what separates them is *excavation*, knowledge computed from artifacts rather than read from narration, with the claims that require runtime logs separating the systems most sharply.

1 Introduction

AI agents are taking on the daily work of enterprise engineers and analysts. The crucial part of this work is the tribal knowledge around it: ask a tenured analyst how October revenue is computed and you get a precise query full of exclusions and joins; ask *why* each exclusion is there, and the pieces of the answer lie scattered across old commits, queries, and dashboards, while the assembled answer lives in one place: their head. An agent without that understanding fails silently, and acquiring tribal knowledge from a company’s artifacts becomes a capability worth measuring.

The knowledge is scattered across the sources companies cannot open up: the codebase and its history, the warehouse, the query log, runtime logs, dashboards; when released, such data is metadata stripped of content (Redset, Snowset [13, 14]), never joinable to the code that produced it. We know this first-hand: in two enterprise deployments of our ingestion system, most recovered knowledge came from artifacts that could never be released; a vendor cannot prove its system works, a buyer cannot compare. As of our survey (July 2026), no benchmark evaluates tribal-knowledge extraction over a company’s full sources, the way coding agents are measured against SWE-bench [8].

Our answer is NovaMart, a simulated e-commerce retailer that is *executed* rather than authored: a conductor replays real shopper traffic through a real application and database for 3.5 months while simulated engineers develop, break, and investigate it. Every row, log line, query, and dashboard

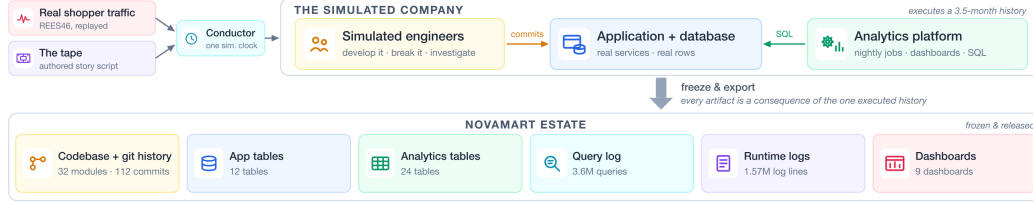


Figure 1: The NovaMart pipeline. The story is scripted; every artifact follows from executing it.

follows from that one executed history: the sources causally agree and the estate is publishable (Figure 1). On top we release 51 audited gold claims, each with a recomputable evidence chain.

Across three widely used agent systems, three runs each, mean recall spans 33%–63%; what separates them is *excavation*, computing knowledge from the data rather than reading it (Section 4).

Contributions. (1) **a simulated company**, an executed world on real traffic, causally consistent, fully released; (2) **51 audited gold claims** scored by a rubric-based LLM judge.

2 The NovaMart Benchmark

2.1 How the world is made

NovaMart is built from two inputs. The first is real shopper behavior: the public event stream from the REES46 dataset [1], replayed hourly as ambient traffic. The second is *the tape*: an authored script of the company’s story, arranged as a causal graph, written by engineers reproducing failure patterns seen in production: a payment provider that delivers every confirmation twice for three hours, a vendor whose weekly price uploads are silently discarded for a month, three teams that each invent their own definition of an active customer. Only the events are authored and their consequences follow from running them: LLM-powered engineers and an analytics platform act out the tape against the live application on one clock, everything a company would accumulate builds up as a side effect, and after 3.5 simulated months the estate is frozen and exported (Figure 1; App. A gives the generator model and one worked incident). The estate is fully inspectable; every claim is re-verifiable without the generator.

2.2 The estate at a glance

Code modules	Git Commits	Warehouse (app)	Warehouse (analytics)	Historical queries	Log entries	Dashboards
32	112	12	24	3,597,650	1,570,017	9

The company is a consumer-electronics retailer that operated through Q4 2019: 81,018 products, 38,950 registered users, about \$2.9M net sales, nightly jobs, monthly statements, a two-generation recommendation engine, and a fraud pipeline; the estate is deliberately small (Section 5).

2.3 The gold claims

A gold claim is one self-contained piece of tribal knowledge in a few plain sentences; for scoring it decomposes into atomic requirements a book must state and statements it must not contradict. Three rules decide membership: the fact is stated plainly in no single artifact; getting it wrong has operational consequences; and it is mechanically recomputable through a typed evidence chain (a SQL statement, a commit hash). Claims were authored by studying the frozen estate, tracing to tape events and to unscripted behavior that emerged in execution. A three-stage audit (blind review, quality review, independent re-derivation of every evidence chain) reduced the 68-claim candidate set to the released 51 (App. B). Each claim is tagged with the sources recovery requires: 49% cannot be recovered from the current code and warehouse alone, and 29% need the query or runtime logs (categories in App. C, an example record in App. D).

The evaluation is built on claims, not questions, because a question presupposes the hardest part of the task: knowing what to ask. Each system explores the estate unprompted and writes a knowledge

document (a “tribal book”), scored following atomic-decomposition practice in factuality evaluation [2, 3]; posed questions would act as pointers and conceal the coverage failures Section 4 reports.

3 Related Work

We divide agentic enterprise benchmarks by how their worlds are made and what they score (Table 1). TheAgentCompany [4] executes real services but borrows its only history from open-source repositories, without warehouse or log surfaces; CRMarena(-Pro) [5] author a CRM organization, consistency asserted by a generator, no codebase; Hex’s DataBench [11] authors warehouse tables with narrated-only history; Spider 2.0 and BIRD [6, 7] pose tasks over real, frozen databases; SWE-bench [8] covers code history only. As of our survey (July 2026), none provides causally consistent sources over a longitudinal enterprise estate, and all grade posed items where ours scores an unprompted knowledge document (Section 2.3); per-cell justification with quotes ships with the release.

4 Experiments

4.1 Setup

We evaluate three widely used agentic systems: Claude Code and Cursor (both running Table 5 at high effort) and Codex (GPT-5.6 Sol at high effort) [12]. Each system receives identical read-only access to the estate and an identical two-part brief, task setup plus end goals (full text in App. E). The brief contains no questions and no pointers at evidence beyond access coordinates. Every system runs as shipped, zero-shot, three times under the same frozen brief, nine books in total. Because we evaluate whole products, differences reflect model and scaffolding together.

4.2 Scoring

Every book is scored per claim by an LLM judge (Gemini 3 Flash preview) against the claim’s rubric, with verdicts correct, partial, missing, or contradicted; the released verdict is the majority over five independent passes. Scoring is stable: a book’s recall moves by at most 5.9 points across passes, usually by 2 or less (bootstrap CIs: App. G). Recall is path-blind: one query or many hops earn the same credit; efficiency is never scored. App. F has an example rubric and the full variance table.

4.3 Results

No system is near saturation. The best system misses over a third of claims, the weakest two thirds.

Reliable knowledge is much smaller than any single run suggests. A single run would have reported Claude Code anywhere from 58.8% to 70.6% [10]; the dependable core, claims solved in all three runs (pass³, [9]), is 35.3% for the best systems (any-run coverage: Table 2). Claude Code and Cursor run the same model; the 3.9-point gap between them is inside the judge spread.

System	Surfaces	Made how?	Longitudinal history	Eval unit
TheAgentCompany	services, chat, files	executed session	in- borrowed OSS repo history, not its own; in-session propagation only	task
CRMarena(-Pro)	CRM org	authored (LLM)	authored timestamps (4-yr span), never executed; no code surface	task
Hex DataBench	warehouse + docs + dbt	authored (gen.)	narrated only	task
Spider 2.0	warehouse + docs + dbt code	real, frozen	none	workflow task
BIRD	databases	real, frozen	none	question
SWE-bench	code + git history	real	real, code only	patch
NovaMart (ours)	code + git, warehouse (app+analytics), runtime logs, query log, BI	executed	3.5 months, all surfaces, cross-surface propagation	claim

Table 1: Agentic enterprise benchmarks by how their worlds are made and what they score.

System	Mean recall	Runs (sorted)	Run spread	Solved in all 3 runs	Solved in any run
Claude Code	62.7%	70.6 / 58.8 / 58.8	11.8	35.3%	84.3%
Cursor	58.8%	62.7 / 56.9 / 56.9	5.9	35.3%	80.4%
Codex	33.3%	35.3 / 33.3 / 31.4	3.9	19.6%	51.0%

Table 2: Claim recall (majority verdict, mean over three runs).

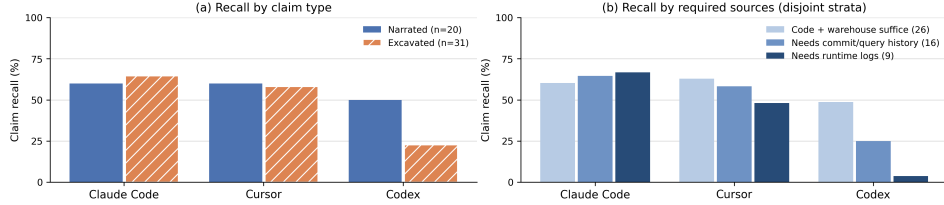


Figure 2: (a) Recall by claim type; (b) by required-source strata: history drops the weakest to 3.7%.

The frontier is unnarrated history. 7 of 51 claims are solved by all nine books, 6 by none; the unsolved require reconstructing history the estate never narrates, like a fee boundary set by a deploy.

4.4 What separates systems

Systems differ most where knowledge must be computed, not read. Each claim is tagged *narrated* (20 claims, rubric satisfiable from text surfaces alone: code, git history, docs) or *excavated* (31, requiring computation over warehouse, logs, or query history). The pooled gap is modest (56.7% vs 48.4%), but Figure 2a separates the systems: the weakest collapses on excavated claims while the two Table 5 systems do not (Table 3). The systems sit within 10 points of each other on narrated claims but span 42 points on excavated ones: what separates these systems is not reading ability but excavation ability.

Claims that require history sources hurt weaker systems most. Figure 2b splits the claims into three disjoint strata: 26 need only current code and warehouse, 16 need commit or query history, 9 need runtime logs. From easiest stratum to hardest, Claude Code gains 6.4 points, Cursor loses 14.7, and Codex collapses by 45.0 (48.7% to 25.0% to 3.7%). Pooled, the strata fall 57.3%, 49.3%, and 39.5%: the artifact classes existing benchmarks lack are exactly where recall is lowest. **The separation is concentrated in the excavated and history-dependent strata, with headroom above the best system; the narrated stratum barely distinguishes the systems.** (Full matrix: App. G.)

5 Limitations, Future Work, and Release

We presented NovaMart: a simulated company, its full estate, and 51 audited claims measuring how much tribal knowledge an agent recovers. The world is small next to a real company, yet far from solved: the best system misses over a third of the claims and none reliably recovers more than 35.3% across its three runs. **The traffic is real and the application, warehouse, dashboards are real running software; what the world lacks is the volume and routine-chore noise of a multi-year history.** Absolute numbers belong to this single world; the narrated-versus-excavated pattern should transfer.

Four further limitations: scores mix model with machinery (we evaluate shipped systems); correctness is decided by an LLM judge with no human-agreement study; prompt sensitivity is unmeasured (the prompt is frozen, App. E); and recall has no precision counterpart: fabricated statements outside the gold claims go unpenalized. **Future work:** a larger world, more simulated companies, and an open reference harness comparing bare models under fixed machinery.

We release everything needed to run, score, and audit the benchmark at <https://novamartbench.com>: the frozen estate (code with git history, warehouse, runtime logs, query log, dashboards), the 51 claims with recomputable evidence chains, the judge harness, and a leaderboard of verified submissions.

References

- [1] REES46. eCommerce behavior data from a multi-category store. Kaggle dataset. <https://www.kaggle.com/datasets/mkechinov/ecommerce-behavior-data-from-multi-category-store>
- [2] S. Min et al. FActScore: Fine-grained atomic evaluation of factual precision in long form text generation. In *EMNLP*, 2023.
- [3] A. Nenkova and R. Passonneau. Evaluating content selection in summarization: The pyramid method. In *HLT-NAACL*, 2004.
- [4] F. F. Xu et al. TheAgentCompany: Benchmarking LLM agents on consequential real world tasks. In *NeurIPS Datasets and Benchmarks*, 2025. arXiv:2412.14161.
- [5] K.-H. Huang et al. CRMarena: Understanding the capacity of LLM agents to perform professional CRM tasks in realistic environments. In *NAACL*, 2025. CRMarena-Pro: arXiv:2505.18878.
- [6] F. Lei et al. Spider 2.0: Evaluating language models on real-world enterprise text-to-SQL workflows. In *ICLR*, 2025. arXiv:2411.07763.
- [7] J. Li et al. Can LLM already serve as a database interface? A big bench for large-scale database grounded text-to-SQLs. In *NeurIPS Datasets and Benchmarks*, 2023. arXiv:2305.03111.
- [8] C. E. Jimenez et al. SWE-bench: Can language models resolve real-world GitHub issues? In *ICLR*, 2024. arXiv:2310.06770.
- [9] S. Yao, N. Shinn, P. Razavi, and K. Narasimhan. τ -bench: A benchmark for tool-agent-user interaction in real-world domains. In *ICLR*, 2025. arXiv:2406.12045.
- [10] E. Miller. Adding error bars to evals: A statistical approach to language model evaluations. arXiv:2411.00640, 2024.
- [11] Hex. Introducing DataBench (<https://hex.tech/blog/databench-agentic-analytics-benchmark/>), and: How we built a lab to evaluate data agents (<https://hex.tech/blog/evaluate-data-agents/>). hex.tech blog, 2026. Accessed 2026-08-29.
- [12] Claude Code (<https://claude.com/claude-code>), Cursor (<https://cursor.com>), and Codex (<https://openai.com/codex>), evaluated as released commercial products.
- [13] A. van Renen et al. Why TPC is not enough: An analysis of the Amazon Redshift fleet. *PVLDB*, 17(11), 2024. Releases Redset: query statistics of 200 provisioned and 200 serverless Redshift instances; no query text or data content.
- [14] M. Vuppapapati et al. Building an elastic query engine on disaggregated storage. In *NSDI*, 2020. Releases Snowset: statistics for ~ 70 M Snowflake queries; no query text or data content.

A A worked propagation trace

Generator disclosure: every simulated-engineer episode in the world’s construction was driven by gpt-5.4-2026-03-05 at medium thinking effort. This is a different model family from the two strongest evaluated systems (both Fable 5); to the extent artifacts are easiest to read for their author’s own family, the advantage would accrue to Codex (GPT-5.6 Sol), the weakest performer, so family bias cannot explain the observed ordering.

This appendix walks through one actual tape incident, exactly as it executed in the simulation. The purchase traffic it runs on is the REES46 stream [1]: a public record of shopper events (product views, cart additions, purchases) from a large multi-category online store, replayed hourly against the application, so every order and payment below traces back to a real browsing session.

The setup: In mid-October 2019 of the simulated history, the order service had no idempotency check: when the payment provider confirmed a purchase, the service created an order and a payment without ever asking whether it had already processed that confirmation. The company’s purchase traffic is the replayed REES46 stream, so the hazard was concrete: deliver any real confirmation twice, and the company books two orders and charges the customer twice.

The authored incident: Knowing the codebase was in this state, the tape scripts a provider glitch: for three hours on the simulated afternoon of 2019-10-14, every purchase confirmation is delivered twice, 5 seconds apart. The tape also scripts the follow-up, set to fire only if the database actually accumulates 10 or more duplicated payment references: the next morning, finance notifies engineering that customers look double-charged and the nightly reconcile has been noisy. That is all that was

authored. From here everything executed end to end; we never touched the database, the codebase, or the logs directly.

Sim time	Surface	What execution produced
Oct 14, 14:00–17:00	Database	17 duplicate orders (and their payments) land, each exactly 5 seconds after its original; no other sub-10-second twin exists anywhere in the company’s history
Oct 15, morning	Tape trigger	the duplicated references push past the follow-up’s threshold of 10, so the follow-up fires: finance reports customers who look double-charged
Oct 15, 14:00	Codebase	a simulated engineer responds: commit b676969 (“Make order callbacks idempotent by payment_ref”) adds the guard, and no new twin is ever created again
after Oct 15 nightly	Database Runtime logs	the storm’s duplicates were never cleaned up; they are still in the frozen table the reconcile job runs every morning and flags the suspect references; it only warns, never fixes, and by the final night it flags 130: the storm twins plus the slower organic repeat purchases

The incident leaves its mark in the suite: gold claim OP-03 tests the scar (payment references are not unique, and the nightly reconcile only warns about the 130 suspect ones), while the story behind it, the storm window, the guard, and the cleanup that never happened, stays recoverable from the commit history and the job logs.

B How the claims were created

The claims were authored by studying the frozen estate itself: reading the git history, querying the warehouse and the logs, and tracing each tape event to its downstream consequences. Many claims capture the intended consequences of scripted events; others capture unscripted behavior that only emerged in execution and was discovered the same way an evaluated system would discover it. A claim enters the gold suite only if it meets three criteria:

1. **Stated plainly nowhere.** The fact appears in no single artifact; reaching it takes computing over the data, reading through history (logs, queries, git), joining evidence across artifacts, or digging out behavior that nothing narrates.
2. **Operationally load-bearing.** Getting it wrong produces wrong numbers, wrong joins, or unsafe changes somewhere in the company’s work.
3. **Mechanically recomputable.** A typed evidence chain (a SQL statement, a commit hash) re-derives it from the shipped artifacts.

Funnel: 68 claims formed the final candidate set (earlier prototype claims, mostly plain surface lookups, had been retired during suite redesigns, hence the non-contiguous claim IDs); each was verified live against the estate before entering the suite (an author-run check, lighter than the independent re-derivation in stage 3). Of the 68, 17 were removed: 10 in a blind review, 5 in a quality review for failing criterion (1) (plain lookups answerable from the current code), and 2 by the independent audit of Section 2.3 (every evidence chain re-derived against the estate), which also corrected the remaining 6 of the 8 claims it found broken (each corrected claim was re-derived again after correction). The released suite is 51 claims. Apart from the mechanical audit, every released claim was reviewed by hand for wording and rubric clarity.

C Claim categories

Every claim carries five tags; the canonical per-claim assignments ship with the release, and the shape labels were human-reviewed. The categories are:

Narration: whether the rubric can be satisfied by reading or only by computing.

Narration	n	Definition
Narrated	20	the rubric can be satisfied from text surfaces alone (current code, git history, repo docs)
Excavated	31	satisfying the rubric requires computing over the data systems (warehouse tables, runtime logs, query history)

Required sources: which historical sources recovery requires. A claim can carry more than one flag; for Figure 2b the flags collapse into three disjoint strata; the query-history and runtime-log flags never co-occur, so 15 distinct claims carry one of the two.

Source flag	n	Disjoint stratum (Figure 2b)	n
Requires commit history	14	Current code and warehouse suffice	26
Requires runtime logs	9	Commit or query history required	16
Requires query history	6	Runtime logs required	9

Claim shape: what kind of fact the claim states.

Claim shape	n	Definition
Computed quantity	12	a number that is stored nowhere and must be calculated fresh from the artifacts
Temporal era	10	what changed inside the company and when, so the same number means different things before and after a date
Absence	9	what looks live but does not exist, never ran, or is dead: no code path, no control arm, a job that silently does nothing
Reconciliation	9	two sources disagree and the claim states which is right and why
Mechanism	6	how the system produces a value or state today, assembled from several places in the current code
Multi-source join	3	the fact only exists once evidence from two or more systems is joined
Other	2	reviewed claims that fit none of the named shapes

Business segment: which part of the company the claim is about.

Business segment	n	Scope
ML & recommendations	15	the recommendation system, its versions, serving, logging, and evaluation
Orders, payments & fraud	11	the order lifecycle, payment and refund handling, fraud checks
Catalog & customers	10	products, pricing, accounts, and the definitions of customers
Revenue & reporting	10	financial statements, revenue definitions, and the reports built on them
Pipelines & scheduling	5	the scheduled batch jobs and what depends on them

Temporal character: how the claim relates to the timeline.

Temporal character	n	Definition
Point-in-time	29	a fact about the frozen state that needs no timeline
Historical reconstruction	19	states what changed and when; recovery requires rebuilding that timeline
Current-state, dated	3	a fact about the current state whose meaning depends on a date it carries

D An example gold claim (FIN-04)

Below is the complete released record of one claim, FIN-04, verbatim (warehouse project id replaced by the App. E placeholder); its rubric is also the judging example of App. F. Every claim in the suite follows this schema: identity and typing, source-requirement flags, the narration, acquisition, and scope tags, the evidence chain (each entry typed by source, carrying the artifact coordinates and the recomputable query or commit), and the match block the judge scores against. The claim-shape,

business-segment, and temporal-character tags of App. C are not in the record; they live in the released category map (category_maps.json), one entry per claim.

```
- claim_id: FIN-04
  claim_name: null
  added_at: '2026-08-19'
  claim_type: dynamic
  claim_text: 'October 2019 has two official gross revenue numbers: the published statement
    says 1,230,332.43
    while the restated view analytics.statements_final says 1,226,764.86. The difference is
    chargebacks
    booked in a separate analytics table and subtracted by the view, never in the
    statements table.'
  source_type: data_warehouse
  requires_commit_history: false
  requires_query_history: false
  requires_runtime_logs: false
  narration: excavated
  acquisition: Hard-derive
  scope:
    customer: novamart
    product_area: Finance
    surface: shared
  evidence:
    - role: primary
      source_type: data_warehouse
      repo: null
      path: null
      span: null
      uri: <warehouse-project>.novamart_analytics.statements_final
      commit_hash: null
      symbol: null
      table: <warehouse-project>.novamart_analytics.statements_final
      columns:
        - month
        - gross
      sql: SELECT s.month, s.gross AS published, f.gross AS restated FROM
        '<warehouse-project>.novamart.statements'
        s JOIN '<warehouse-project>.novamart_analytics.statements_final' f USING (month) WHERE
        s.month = '2019-10'
      excerpt: Returns published=1230332.43, restated=1226764.86 (delta 3567.57 = booked
        chargebacks).
    match:
      must_include:
        - The published October 2019 statement gross is 1,230,332.43.
        - The chargeback-restated October gross (statements_final) is 1,226,764.86.
        - Chargebacks are booked in an analytics table/view and are not part of the statements
          table.
      must_not_include:
        - There is only one official October gross figure.
      rubric: Correct if the book states the semantic requirements in must_include and avoids
        must_not_include
        contradictions.
```

E The elicitation prompt (infrastructure identifiers replaced by placeholders)

Every run was launched with the same single message, reproduced verbatim:

```
Execute this task as mentioned at '~/book_runs/prompts/novamart_sim_goal_context.md'. MUST
follow the extra context at '~/book_runs/prompts/novamart_sim_extra_context.md'
DO NOT deviate from the task and follow it line by line as mentioned in the doc.
```

The two files it references are frozen as part of the benchmark and are reproduced below in full, in their original markdown. Five infrastructure identifiers are replaced by placeholders: the machine

paths by <workspace> and <access-pack>, the warehouse project id by <warehouse-project>, the GitHub organization by <github-org>, and the local dashboard URL by <redash-url>.

novamart_sim_goal_context.md:

Novamart Tribal Knowledge Goal Context

Goal

Accomplish the following end goals:

Phase 1: Understand how the business numbers are actually produced

- Understand how novamart computes its core financial numbers end to end: orders, payments, refunds, monthly revenue, and the finance statements.
- Identify which tables, jobs, and dashboards are treated as the source of truth for each number, and how the definitions have changed over the company's history.
- Final Goal: Be able to answer "what was revenue in a given month, and why" the way a tenured finance/data person at the company would.

Phase 2: Understand product and customer analytics

- Understand how product performance is reported: best sellers, brand and category revenue, and the assumptions baked into those reports.
- Understand how the company counts its customers: signups, active customers, and any other customer segments in use and which definition each consumer (dashboards, jobs, leadership reporting) actually uses.
- Final Goal: Be able to explain any number on the company's dashboards, including when a dashboard's number should not be trusted at face value.

Phase 3: Understand the ML systems and scheduled jobs

- Understand any machine-learning or recommendation systems the company runs: how their outputs are produced, served, and logged, and how they have evolved across versions.
- Understand the scheduled batch jobs: what each one produces, what breaks when they fail, and how their outputs feed reporting and serving.
- Final Goal: Be able to judge whether the company's ML systems actually work, and to safely modify or extend the batch pipelines.

Tribal Knowledge Requirement

The tribal knowledge document should be comprehensive enough to support executing these goals.

novamart_sim_extra_context.md:

Novamart Extra Context

Task

[TASKID: novamart] You are a senior software engineer and your task is to help me understand the tribal knowledge from my company's systems: the codebase, the data warehouse, and the dashboards.

Data Sources

- Codebase: GitHub, repo cloned and checked out at '<workspace>/novamart' (pinned at commit '2ae79e2'); you must only use this repo. A read-only GitHub token (org '<github-org>') is stored at '<access-pack>/agent-github-pat' if you need remote git operations.
- Data Warehouse: BigQuery, GCP project '<warehouse-project>', read-only service-account credentials at '<access-pack>/sa-key.json'. Datasets: 'novamart' (app tables), 'novamart_analytics' (analytics tables + views), 'novamart_logs' (log exports, including the historical query log 'db_queries').
- Dashboards: Redash, available at '<redash-url>' (read-only). Credentials and API key are stored at '<access-pack>/redash-agent-creds'.

Instructions

Your understanding MUST only come from the searching and understanding you do on the fly and not using any external knowledge you find in the machine.

Starting: When starting this exercise you must print starting time in wall clock time and then print a random uuid string.

Don'ts

- DO NOT MUTATE OR DELETE ANY DATA anywhere. Run SELECT and other read-only queries only. Do not create, edit, archive, or refresh anything in Redash.
- Do not modify the repository or push via the GitHub token.

Output

- Output MUST be in a single markdown file.
- It is a MUST thing that the output surfaces these 8 pillars on top, in this numbered order. Extra detail lives in an appendix.
 1. Summary
 2. Why this project
 3. Business understanding
 4. Metrics
 5. System
 6. Data
 7. Experimentation
 8. Glossary
- Every claim in the document must cite its evidence: a commit hash, table/view name, query, log line, or dashboard.
- All outputs and intermediate outputs MUST and only be stored in the sub-directory: create a sub-directory named with the uuid you printed at start and dump everything there, including the final markdown file.

Time Limit

You don't have any time limit on it, take as much time as needed.

A note on scope: the brief names goal areas rather than answers, the way a real onboarding brief tells a newcomer what the job covers. Some goal areas overlap the claim taxonomy at the category level; the customer-counting goal, for example, sits in the same territory as the tape's active-customer-definitions incident (Section 2.1). The brief never names an incident, an artifact, or any claim's content; recovering the specifics is the system's task.

All nine runs were performed on August 27, 2026, on Claude Code 2.1.252, Cursor 3.15.6, and Codex CLI 0.120.0 (GPT-5.6 Sol); the released books freeze their outputs.

F Judge rubric and stability

Each claim's rubric lists statements a book must contain and statements it must not contradict; the judge assigns correct, partial, missing, or contradicted per claim, and the released verdict is the majority over five independent judge passes (ties resolve to the worse verdict; no five-pass tie occurred in the released runs, 0 of 459 claim-book cells). Claim recall is the fraction of the 51 claims whose majority verdict is correct; partial, missing, and contradicted earn no credit, and a claim counts as solved in a run when its majority verdict is correct.

Example rubric (FIN-04), the statements a book must include:

Rubric item	Required statement
Published figure	October 2019 gross revenue as published is 1,230,332.43
Restated figure	the restated view reports 1,226,764.86
Mechanism	chargebacks are booked in an analytics table/view outside the statements table

Per-run stability (claim recall per judge pass, five judge passes per run):

Run	Pass recalls (%)	Spread
Claude Code r1	72.5 / 70.6 / 70.6 / 70.6 / 72.5	2.0
Claude Code r2	58.8 / 60.8 / 58.8 / 58.8 / 58.8	2.0
Claude Code r3	58.8 / 58.8 / 56.9 / 58.8 / 56.9	2.0
Cursor r1	56.9 / 51.0 / 56.9 / 56.9 / 56.9	5.9
Cursor r2	56.9 / 56.9 / 56.9 / 56.9 / 56.9	0.0
Cursor r3	62.7 / 62.7 / 62.7 / 62.7 / 62.7	0.0
Codex r1	33.3 / 33.3 / 35.3 / 33.3 / 33.3	2.0
Codex r2	29.4 / 31.4 / 31.4 / 31.4 / 27.5	3.9
Codex r3	31.4 / 35.3 / 35.3 / 35.3 / 35.3	3.9

G Per-claim scores

Table 5 lists the extremes of the suite. Table 6 gives the released verdict for every claim and every run: the majority over five judge passes, ties resolved to the worse verdict, computed from the released scorecards; Y correct, ~ partial, . missing, X contradicted, rows sorted hardest-first by the number of runs solving the claim. Tables 7–9 break recall down by the categories of App. C. Claim-level bootstrap 95% confidence intervals for Table 2’s mean recall (10,000 resamples of the 51 claims with replacement, seed 0, recomputable from the released verdicts): Claude Code 62.7 [52.9, 72.5], Cursor 58.8 [48.4, 69.3], Codex 33.3 [22.2, 44.4]. The Claude Code and Cursor intervals overlap almost entirely, consistent with Section 4; Codex is separated from both. Run spreads in Table 2 are computed before rounding (Cursor: 5.88, displayed as 5.9). Tables 7–9 slice the 51 claims into groups of 2–26 and inherit correspondingly wider intervals; read them as descriptive. The nine books span 5,000–8,490 words; length does not predict recall ($r = -0.18$ across the nine books), and the longest book belongs to the weakest system.

Narration	n	Claude Code	Cursor	Codex	Pooled
Narrated	20	36/60 = 60.0	36/60 = 60.0	30/60 = 50.0	102/180 = 56.7
Excavated	31	60/93 = 64.5	54/93 = 58.1	21/93 = 22.6	135/279 = 48.4

Table 3: Recall by narration: correct claim-runs / total (%).

Required-source stratum	n	Claude Code	Cursor	Codex	Pooled
Code + warehouse suffice	26	47/78 = 60.3	49/78 = 62.8	38/78 = 48.7	134/234 = 57.3
Needs commit or query history	16	31/48 = 64.6	28/48 = 58.3	12/48 = 25.0	71/144 = 49.3
Needs runtime logs	9	18/27 = 66.7	13/27 = 48.1	1/27 = 3.7	32/81 = 39.5

Table 4: Recall by required-source stratum: correct claim-runs / total (%).

The runtime-log stratum comprises HD-02, NB-21, NB-24, NB-25, NB-30, OP-04, RS-03, RS-07, and XW-03; together with Table 6, Tables 3 and 4 make every narration and stratum figure in the paper recomputable in-paper.

Item-level summary	Count	Claims
Solved by all nine runs	7	CU-03A, HD-01, MLE-01, NB-10, OP-02, OP-03, RS-04
Solved by no run of any system	6	NB-15, NB-30, OP-04, PR-03, XM-02, XM-04

Table 5: The extremes of the suite.

Table 6: Claim-by-run verdict matrix (51 claims $\times$ 9 runs, majority verdicts, hardest-first).

Claim	#Y	cc_r1	cc_r2	cc_r3	cur_r1	cur_r2	cur_r3	cx_r1	cx_r2	cx_r3
NB-15	0/9	~	~	~	~	.	~	.	.	~
NB-30	0/9	.	.	.	.	.	.	.	.	.
OP-04	0/9	~	~	~	~	~	~	~	~	~
PR-03	0/9	.	.	.	.	~	~	.	.	.
XM-02	0/9	~	~	~	~	~	~	.	.	~

Claim	#Y	cc_r1	cc_r2	cc_r3	cur_r1	cur_r2	cur_r3	cx_r1	cx_r2	cx_r3
XM-04	0/9	.	.	.	.	.	.	.	.	.
NB-28	1/9	.	Y	.	~	.	.	.	.	.
NB-29	1/9	Y	.	~	X	.	~	.	.	.
XW-03	1/9	Y	~	~	~	~	~	.	.	.
DB-01	2/9	.	.	.	.	.	Y	Y	.	.
MLD-02	2/9	Y	.	.	.	.	.	Y	.	~
NB-18	2/9	~	~	Y	~	Y	~	.	.	.
XM-03	2/9	~	Y	~	~	~	Y	~	.	.
XW-05	2/9	~	.	~	Y	Y	~	.	.	.
HB-01	4/9	Y	~	Y	~	Y	Y	.	.	.
HB-02	4/9	Y	~	Y	Y	~	Y	.	.	.
MLB-03	4/9	Y	~	Y	~	Y	~	~	~	Y
NB-21	4/9	Y	Y	Y	~	Y	~	~	.	~
NB-24	4/9	Y	Y	Y	Y	.	~	.	~	.
PR-01	4/9	Y	~	~	Y	~	Y	~	Y	~
RS-01	4/9	Y	~	Y	Y	~	~	~	~	Y
XW-04	4/9	Y	Y	~	~	Y	~	~	~	Y
DB-02	5/9	~	Y	Y	Y	~	Y	~	Y	~
FIN-04	5/9	Y	~	Y	~	~	Y	~	Y	Y
HC-01	5/9	~	Y	Y	Y	Y	~	Y	~	~
HC-02	5/9	Y	Y	Y	~	Y	Y	.	.	~
MLD-01	5/9	Y	Y	Y	Y	~	.	Y	~	~
NB-02	5/9	Y	Y	Y	~	Y	Y	~	.	~
NB-03	5/9	Y	Y	Y	~	Y	Y	.	.	~
NB-05	5/9	Y	Y	Y	~	Y	Y	~	~	~
NB-22	5/9	Y	Y	~	Y	Y	Y	.	~	~
RS-02	5/9	~	Y	Y	Y	Y	Y	~	~	~
RS-03	5/9	Y	Y	~	Y	~	Y	~	Y	~
XW-02	5/9	Y	.	.	Y	Y	Y	Y	.	.
HD-02	6/9	Y	Y	Y	Y	Y	Y	.	.	.
NB-11	6/9	~	Y	Y	Y	~	Y	.	Y	Y
NB-25	6/9	Y	Y	Y	Y	Y	Y	~	~	~
RS-07	6/9	Y	Y	Y	Y	Y	Y	.	.	.
CU-04	7/9	Y	Y	~	Y	Y	Y	Y	~	Y
NB-09	7/9	Y	Y	X	Y	X	Y	Y	Y	Y
XM-01	7/9	Y	.	Y	Y	Y	Y	Y	X	Y
FR-01	8/9	Y	Y	Y	Y	Y	Y	~	Y	Y
HB-06	8/9	Y	Y	~	Y	Y	Y	Y	Y	Y
RS-06	8/9	Y	~	Y	Y	Y	Y	Y	Y	Y
CU-03A	9/9	Y	Y	Y	Y	Y	Y	Y	Y	Y
HD-01	9/9	Y	Y	Y	Y	Y	Y	Y	Y	Y
MLE-01	9/9	Y	Y	Y	Y	Y	Y	Y	Y	Y
NB-10	9/9	Y	Y	Y	Y	Y	Y	Y	Y	Y
OP-02	9/9	Y	Y	Y	Y	Y	Y	Y	Y	Y
OP-03	9/9	Y	Y	Y	Y	Y	Y	Y	Y	Y
RS-04	9/9	Y	Y	Y	Y	Y	Y	Y	Y	Y

Claim shape	n	Claude Code	Cursor	Codex	Pooled
Absence	9	74.1	70.4	48.1	64.2
Reconciliation	9	59.3	59.3	40.7	53.1
Mechanism	6	61.1	55.6	27.8	48.1
Temporal era	10	53.3	43.3	23.3	40.0
Computed quantity	12	55.6	50.0	13.9	39.8
Multi-source join	3	77.8	88.9	77.8	81.5
Other	2	100.0	100.0	50.0	83.3

Table 7: Claim recall (%) by claim shape, averaged over three runs per system; the Pooled column averages all nine runs.

H LLM usage

The simulation’s engineer model is disclosed in App. A and the judge model in Section 4.2. In addition, the authors used an LLM assistant for editing and formatting this paper. The benchmark

Business segment	n	Claude Code	Cursor	Codex	Pooled
Orders, payments & fraud	11	84.8	78.8	48.5	70.7
ML & recommendations	15	60.0	53.3	35.6	49.6
Revenue & reporting	10	56.7	63.3	26.7	48.9
Catalog & customers	10	60.0	50.0	26.7	45.6
Pipelines & scheduling	5	40.0	40.0	20.0	33.3

Table 8: Claim recall (%) by business segment, averaged over three runs per system.

Temporal character	n	Claude Code	Cursor	Codex	Pooled
Point-in-time	29	60.9	62.1	43.7	55.6
Historical reconstruction	19	61.4	54.4	21.1	45.6

Table 9: Claim recall (%) by temporal character, averaged over three runs per system. The smallest group (current-state, dated; 3 claims) is omitted.

design, the authoring and audit of the claims, the experiments, the analysis, and every statement in the text are the authors’ own; the authors verified every number and reference.